In [2]:

```
#required libraries
```

In [1]:

```
import torch
import cv2
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

from torchvision import models, transforms

from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget
```

In [ ]:

```
#loading trained model
```

In [3]:

```
# Define device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Recreate model architecture (same as during training)
model = models.resnet50(weights=None)  # no pretrained weights needed
num_features = model.fc.in_features
model.fc = torch.nn.Sequential(
    torch.nn.Dropout(0.3),
    torch.nn.Linear(num_features, 4)
)

# Load your trained weights
model.load_state_dict(torch.load("mri_model_final.pth", map_location=device))
model = model.to(device)
model.eval()

# Class names (must match training order)
classes = ['glioma', 'meningioma', 'notumor', 'pituitary']
print("Model loaded successfully!")
```

```
Using device: cuda
Model loaded successfully!
```

In [ ]:

```
#Defining Image transform
```

In [5]:

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

In [ ]:

```
#Load MRI image
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In [13]:

```
import os
from PIL import Image
import numpy as np
import random

folder = "C:/Users/sampl/Desktop/Desktop/Neurovision_mri/Data/Testing/glioma"

image_file = random.choice(os.listdir(folder))
test_image_path = os.path.join(folder, image_file)

print("Using image:", image_file)

img = Image.open(test_image_path).convert("RGB")
input_tensor = transform(img).unsqueeze(0).to(device)

img_resized = img.resize((224, 224))
rgb_img = np.array(img_resized) / 255.0
```

```
Using image: Te-gl_184.jpg
```

In [ ]:

```
#Get Model Prediction
```

In [14]:

```
with torch.no_grad():
    outputs = model(input_tensor)
    probabilities = torch.softmax(outputs, dim=1)
    pred_class = torch.argmax(probabilities).item()
    confidence = probabilities[0][pred_class].item()

print(f"Prediction: {classes[pred_class]}")
print(f"Confidence: {confidence:.2%}")
```

```
Prediction: glioma
Confidence: 77.11%
```

In [ ]:

```
#Select target layer
```

In [16]:

```
target_layers = [model.layer4[-1]]
```

In [17]:

```
#Generate GradCAM Heatmap
```

In [19]:

```
# Create GradCAM object
cam = GradCAM(model=model, target_layers=target_layers)

# Specify the target class (the predicted one)
targets = [ClassifierOutputTarget(pred_class)]

# Generate heatmap
grayscale_cam = cam(input_tensor=input_tensor, targets=targets)
grayscale_cam = grayscale_cam[0]  # remove batch dimension
```

In [ ]:

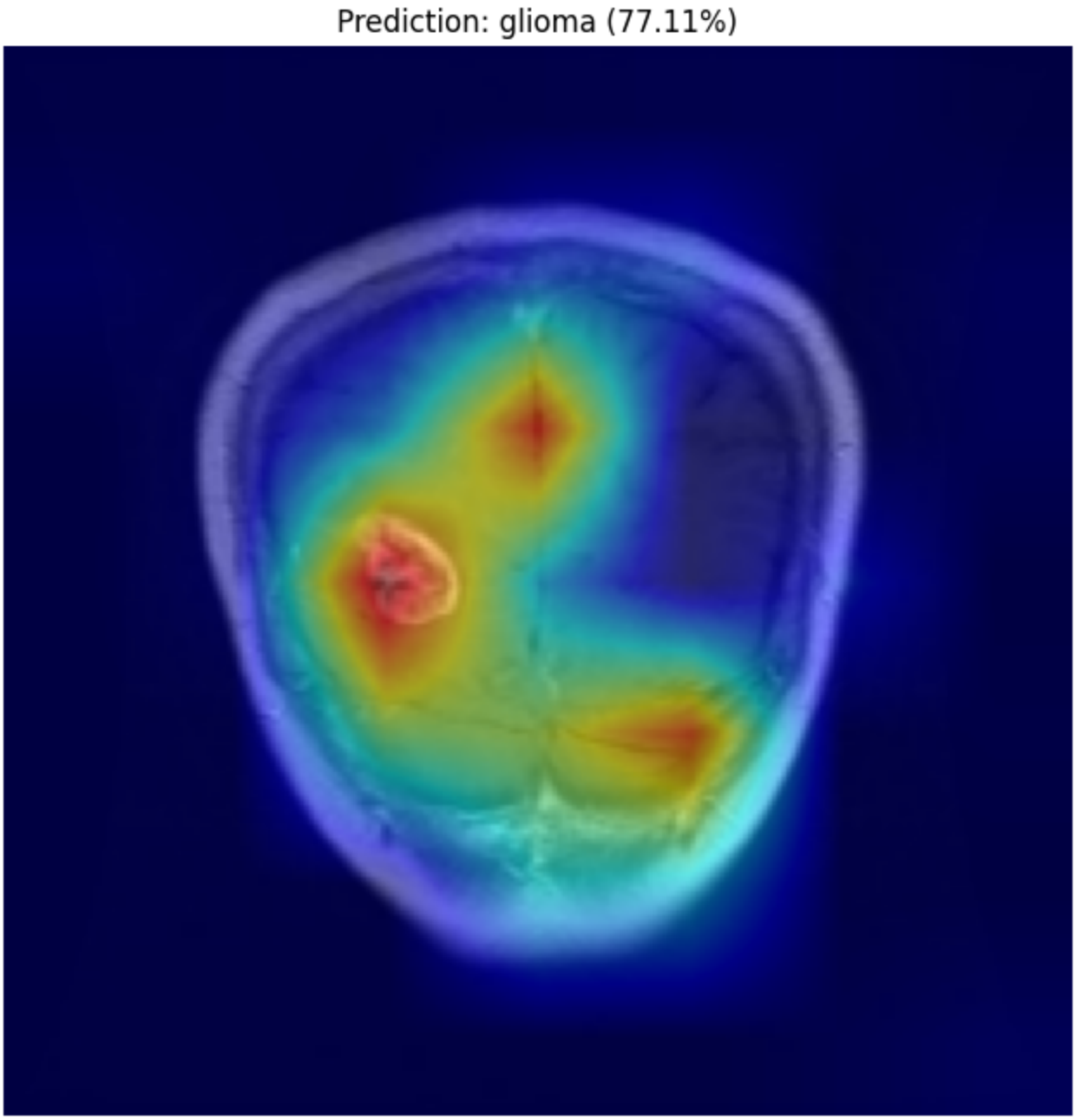
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
#Overlay Heatmap on MRI
```

In [20]:

```
# Convert grayscale heatmap to color overlay and combine with image
visualization = show_cam_on_image(rgb_img, grayscale_cam, use_rgb=True)

# Display the result
plt.figure(figsize=(8, 8))
plt.imshow(visualization)
plt.axis('off')
plt.title(f"Prediction: {classes[pred_class]} ({confidence:.2%})")
plt.show()
```



In [ ]:

```
#SAVE RESULT
```

In [21]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
output_path = "gradcam_result.jpg"
cv2.imwrite(output_path, cv2.cvtColor(visualization, cv2.COLOR_RGB2BGR))
print(f"Heatmap saved to {output_path}")
```

```
Heatmap saved to gradcam_result.jpg
```

In [ ]:

```

```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF